

WPF Image Viewer Service

System

You are a senior C# developer.

Target: .NET 8, WPF MVVM, Autofac DI.

Write production code.

Avoid pseudo code.

Answer only in Russian, where possible.

User

создай сервис для управления просмотром изображений.

основные требования: зуммирование, перемещение по изображению, возможность встройки изображения в размер окна, переход к следующему изображению, переход к предыдущему изображению, удаление текущего изображения

Assistant

Для реализации такого функционала в WPF с использованием MVVM и Autofac, нам нужно разделить логику на три части: **Service** (управление данными), **ViewModel** (управление состоянием UI) и **View** (отрисовка).

Для реализации зума и перемещения (Pan & Zoom) в WPF стандартными средствами лучше всего использовать `RenderTransform`.

1. Модель данных

```
public record ImageItem(string FilePath, string Title);
```

2. Сервис управления изображениями (IImageService)

Сервис отвечает только за работу с коллекцией и навигацию.

```
public interface IImageService
{
    IEnumerable<ImageItem> GetImages();
    void DeleteCurrentImage(ImageItem currentImage);
    void Next();
    void Previous();
}
```

```

    void SetCurrentImage(ImageItem image);
    ImageItem CurrentImage { get; }
    bool HasNext { get; }
    bool HasPrevious { get; }
}

public class ImageService : IImageService
{
    private readonly List<ImageItem> _images;
    private int _currentIndex = 0;

    public ImageService()
    {
        // В реальном приложении здесь будет загрузка из папки
        _images = new List<ImageItem>
        {
            new ("image1.jpg", "Первое фото"),
            new ("image2.jpg", "Второе фото"),
            new ("image3.jpg", "Третье фото")
        };
    }

    public IEnumerable<ImageItem> GetImages() => _images.AsReadOnly();
    public ImageItem CurrentImage => _images.ElementAtOrDefault(_currentIndex);
    public bool HasNext => _currentIndex < _images.Count - 1;
    public bool HasPrevious => _currentIndex > 0;

    public void SetCurrentImage(ImageItem image)
    {
        _currentIndex = _images.IndexOf(image);
    }

    public void Next()
    {
        if (HasNext) _currentIndex++;
    }

    public void Previous()
    {
        if (HasPrevious) _currentIndex--;
    }

    public void DeleteCurrentImage(ImageItem currentImage)
    {
        if (currentImage != null)
        {
            _images.Remove(currentImage);
            if (_currentIndex >= _images.Count) _currentIndex = _images.Count - 1;
            if (_currentIndex < 0) _currentIndex = 0;
        }
    }
}

```

3. ViewModel

Здесь мы используем `TransformGroup` для реализации масштабирования и перемещения.

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;

public partial class ImageViewerViewModel : ObservableObject
{
    private readonly IImageService _imageService;

    [ObservableProperty] private ImageItem? _currentImage;
    [ObservableProperty] private double _zoomLevel = 1.0;
    [ObservableProperty] private double _offsetX = 0;
    [ObservableProperty] private double _offsetY = 0;

    // Свойства для UI
    public bool CanNext => true; // В реальности привязать к HasNext из сервиса

    public ImageViewerViewModel(IImageService imageService)
    {
        _imageService = imageService;
        _currentImage = _imageService.CurrentImage;
    }

    [RelayCommand]
    public void Next()
    {
        _imageService.Next();
        UpdateCurrentImage();
    }

    [RelayCommand]
    public void Previous()
    {
        _imageService.Previous();
        UpdateCurrentImage();
    }

    [RelayCommand]
    public void Delete()
    {
        var current = CurrentImage;
        _imageService.DeleteCurrentImage(current);
        UpdateCurrentImage();
    }

    [RelayCommand]
    public void ZoomIn() => ZoomLevel = Math.Min(ZoomLevel + 0.2, 5.0);

    [RelayCommand]
    public void ZoomOut() => ZoomLevel = Math.Max(ZoomLevel - 0.2, 0.1);

    [RelayCommand]
    public void ResetTransform()
    {
        ZoomLevel = 1.0;
    }
}
```

```

        OffsetX = 0;
        OffsetY = 0;
    }

    [RelayCommand]
    public void FitToWindow()
    {
        ZoomLevel = 1.0;
        OffsetX = 0;
        OffsetY = 0;
    }

    private void UpdateCurrentImage()
    {
        CurrentImage = _imageService.CurrentImage;
        // Сбрасываем трансформацию при смене фото
        ResetTransform();
    }
}

```

4. View (XAML)

Для реализации "Pan" (перемещения) мы привяжем `RenderTransform` к свойствам `OffsetX` и `OffsetY`. Для полноценного Drag-and-Drop перемещения в production-коде обычно добавляют обработку событий `MouseMove` и `MouseWheel`.

```

<Window ...
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:vm="clr-namespace:YourApp.ViewModels"
    Title="Image Viewer" Height="600" Width="800">

    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/> <!-- Toolbar -->
            <RowDefinition Height="*/> <!-- Image Area -->
            <RowDefinition Height="Auto"/> <!-- Status -->
        </Grid.RowDefinitions>

        <!-- Toolbar -->
        <StackPanel Orientation="Horizontal" Background="#F0F0F0" Padding="5">
            <Button Content="Prev" Command="{Binding PreviousCommand}" Margin="5,0"/>
            <Button Content="Next" Command="{Binding NextCommand}" Margin="5,0"/>
            <Separator Margin="5,0"/>
            <Button Content="Zoom In" Command="{Binding ZoomInCommand}" Margin="5,0"/>
            <Button Content="Zoom Out" Command="{Binding ZoomOutCommand}" Margin="5,0"/>
            <Button Content="Reset" Command="{Binding ResetTransformCommand}" Margin="5,0"/>
            <Button Content="Fit" Command="{Binding FitToWindowCommand}" Margin="5,0"/>
            <Separator Margin="5,0"/>
            <Button Content="Delete" Command="{Binding DeleteCommand}" Background="LightCoral"/>
        </StackPanel>

        <!-- Image Viewer Area -->
        <ScrollView Grid.Row="1" HorizontalScrollBarVisibility="Auto"

```

```

VerticalScrollBarVisibility="Auto"&gt;
    &lt;Grid ClipToBounds="True"&gt;
        &lt;Image Source="{Binding CurrentImage.FilePath}" Stretch="Uniform"&gt;
            &lt;Image.RenderTransform&gt;
                &lt;TransformGroup&gt;
                    &lt;ScaleTransform ScaleX="{Binding ZoomLevel}" ScaleY="{Binding
ZoomLevel}"/&gt;
                    &lt;TranslateTransform X="{Binding OffsetX}" Y="{Binding OffsetY}"/&gt;
                &lt;/TransformGroup&gt;
            &lt;/Image.RenderTransform&gt;
        &lt;/Image&gt;
    &lt;/Grid&gt;
&lt;/ScrollView&gt;

&lt;!-- Status Bar --&gt;
&lt;StatusBar Grid.Row="2"&gt;
    &lt;TextBlock Text="{Binding CurrentImage.Title, FallbackValue='No Image selected'}"/&gt;
&lt;/StatusBar&gt;
&lt;/Grid&gt;
&lt;/Window&gt;

```

5. Регистрация в Autofac (App.xaml.cs)

```

public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        var builder = new ContainerBuilder();

        // Регистрация сервисов как Singletons, чтобы состояние сохранялось
        builder.RegisterType<ImageService>().As<IImageService>().SingleInstance();

        // Регистрация ViewModel
        builder.RegisterType<ImageViewerViewModel>();

        var container = builder.Build();

        var mainWindow = new MainWindow();
        var vm = container.Resolve<ImageViewerViewModel>();
        mainWindow.DataContext = vm;

        mainWindow.Show();
        base.OnStartup(e);
    }
}

```

Ключевые моменты реализации:

1. **Разделение ответственности:** `ImageService` ничего не знает о UI, он просто хранит список и управляет индексом. Это позволяет легко протестировать логику (Unit Tests).

2. **Трансформации:** Использование `TransformGroup` в XAML позволяет одновременно масштабировать и двигать изображение.
3. **Производительность:** Для работы с очень большими изображениями в продакшене рекомендуется использовать `BitmapCache` или загружать изображения асинхронно.
4. **MVVM Toolkit:** Я использовал `CommunityToolkit.Mvvm` (современный стандарт для .NET 8), который генерирует boilerplate код (`RelayCommand`, `ObservableProperty`) через `Source Generators`.

User

`public partial class ImageViewerViewModel : ObservableObject`
может лучше перенести свойства зависимостей в `Behaviors` нежели осуществлять такое наследование

Assistant

Это отличный вопрос, демонстрирующий глубокое понимание чистоты архитектуры. Вы правы в том, что **ViewModel не должна зависеть от UI-фреймворка** (в данном случае от `ObservableObject` из WPF/CommunityToolkit, если мы говорим о "чистой" архитектуре).

Однако здесь важно разделять два понятия:

1. **Наследование от `ObservableObject`** : Это не зависимость от UI-библиотек в контексте логики, а использование инфраструктуры для уведомления о свойствах (INPC).
2. **Перенос логики в Behaviors**: Behaviors (Поведения) предназначены для обработки **интерактивности** (клики, свайпы, прокрутка колесика), а не для хранения состояния.

Давайте разберем, как сделать архитектуру по-настоящему профессиональной.

1. Где должен жить код?

| Функция | Где должна быть | Почему? |

2. **Состояние** (`ZoomLevel`, `Offset`) | **ViewModel** | Это "состояние экрана". Если вы захотите сохранить положение картинки при выходе из программы, вы сохраните поля из `ViewModel`, а не из `Behavior`.
3. **Интерактив** (`MouseWheel` -> `Zoom`, `MouseMove` -> `Pan`) | **Behaviors** | `ViewModel` не должна знать, что пользователь крутит колесико мыши. Она должна просто вызвать команду `ZoomInCommand` .

2. Правильная реализация (Hybrid Approach)

Мы оставим `ViewModel` как "мозг", а `Behavior` сделаем "руками".

Шаг A: Чистая ViewModel (только логика)

Она по-прежнему должна уведомлять об изменениях, иначе UI не узнает, что картинка изменилась.

```
// ViewModel остается "мозгом". Она не знает о мышке, она знает о командах.
public partial class ImageViewerViewModel : ObservableObject
{
    private readonly IImageService _imageService;

    [ObservableProperty] private double _zoomLevel = 1.0;
    [ObservableProperty] private double _offsetX = 0;
    [ObservableProperty] private double _offsetY = 0;

    [RelayCommand]
    public void Zoom(double delta)
    {
        // Логика расчета зума через дельту (из поведения)
        ZoomLevel = Math.Clamp(ZoomLevel + delta, 0.1, 10.0);
    }

    [RelayCommand]
    public void Pan(double dx, double dy)
    {
        OffsetX += dx;
        OffsetY += dy;
    }
}
```

Шаг Б: Behavior (Прослойка взаимодействия)

Создаем Behavior, который переводит события мыши в команды ViewModel.

```
using Microsoft.Xaml.Behaviors;
using System.Windows.Input;

public class ImageInteractionBehavior : Behavior<Image>
{
    // Привязываем команды из ViewModel
    public static readonly DependencyProperty ZoomCommandProperty =
        DependencyProperty.Register("ZoomCommand", typeof ICommand), typeof(ImageInteractionBehavior),
        new PropertyMetadata(null));

    public ICommand ZoomCommand
    {
        get => (ICommand)GetValue(ZoomCommandProperty);
        set => SetValue(ZoomCommandProperty, value);
    }

    protected override void OnAttached()
    {
        base.OnAttached();
        this.MouseDown += OnMouseDown;
        this.MouseWheel += OnMouseWheel;
    }
}
```

```

private void OnMouseWheel(object sender, MouseEventArgs e)
{
    if (ZoomCommand is ICommand command)
    {
        // Передаем направление прокрутки как параметр
        double delta = e.Delta > 0 ? 0.1 : -0.1;
        command.Execute(delta);
    }
}

private void OnMouseDown(object sender, MouseButtonEventArgs e)
{
    // Здесь будет логика захвата мыши для Pan (перемещения)
}
}

```

Шаг В: Использование в XAML

Теперь View не просто привязана к свойствам, она "слушает" поведение.

```

<Image Source="{Binding CurrentImage.FilePath}">
    <!-- 1. Трансформации (Связь Состояние -> Визуализация) -->
    <Image.RenderTransform>
        <TransformGroup>
            <ScaleTransform ScaleX="{Binding ZoomLevel}" ScaleY="{Binding ZoomLevel}" />
            <TranslateTransform X="{Binding OffsetX}" Y="{Binding OffsetY}" />
        </TransformGroup>
    </Image.RenderTransform>

    <!-- 2. Поведения (Связь Событие -> Команда ViewModel) -->
    <i:Interaction.Behaviors>
        <local:ImageInteractionBehavior ZoomCommand="{Binding ZoomCommand}" />
    </i:Interaction.Behaviors>
</Image>

```

Итоговый вердикт senior-разработчика:

1. Наследование от `ObservableObject` оставляем. Это стандарт для MVVM. Без него вам придется вручную реализовывать `INotifyPropertyChanged` в каждом классе, что превратит код в спагетти. Это не "грязный" код, это использование паттерна.
2. Свойства (`Zoom`, `Offset`) НЕЛЬЗЯ переносить в `Behaviors`. Если вы сделаете это, ваша `ViewModel` станет "пустой" и бесполезной. Вы не сможете написать Unit-тест на проверку того, что при зуме картинка не вылетает за пределы допустимых значений, потому что логика будет заперта внутри UI-компонента (`Behavior`).
3. `Behaviors` используем только для "перевода". `Behavior` — это переводчик с языка событий Windows (`MouseWheel`, `Touch`) на язык команд вашей `ViewModel`.

Золотое правило:

- **ViewModel** владеет **данными и правилами** (какой может быть зум).
- **Behavior** владеет **взаимодействием** (как именно пользователь меняет зум).
- **View** владеет **представлением** (как это выглядит на экране).